

Compilador JavaSubset $\rightarrow$ x86-64

con Verificación Formal (Hoare + Z3)

Fases 1–5

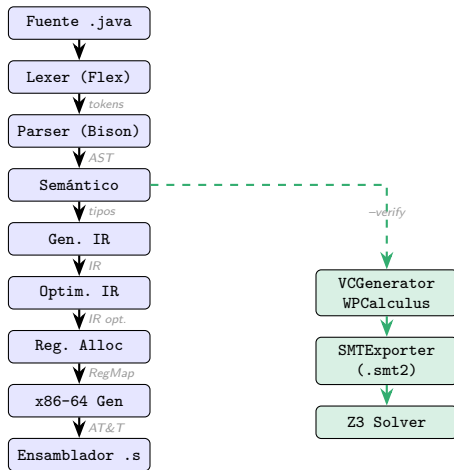
nickhernd

Diseño de Compiladores

28 de junio de 2026

- 1 Arquitectura general
- 2 Front-end: Léxico y Sintáctico
- 3 Análisis semántico
- 4 Representación Intermedia
- 5 Backend x86-64
- 6 Asignación de registros
- 7 Verificación formal
- 8 Pruebas y resultados
- 9 Conclusiones

Pipeline del compilador



Subconjunto Java soportado

Construcciones

- Tipos: `int`, `double`, `boolean`
- Variables locales + métodos estáticos
- Asignación, aritmética, comparación
- `if/else`, `while`
- `return`, llamadas a métodos
- `System.out.println`

Anotaciones de verificación (Fase 5)

<code>@pre expr</code>	Precondición
<code>@post expr</code>	Postcondición
<code>@invariant expr</code>	Invariante de bucle
<code>@variant expr</code>	Terminación
<code>==></code>	Implicación
<code>result</code>	Valor de retorno

Ejemplo

```
@pre true
@post result >= 0
public static int abs(){...}
```

Analizador léxico (Flex)

Tokens principales

Reservadas exactas (if, while, return,...)

Identificadores [a-zA-Z][a-zA-Z0-9]*

Enteros [0-9]+

Reales [0-9]+\.[0-9]+

Anotaciones @pre, @post, @invariant

Reglas especiales — Fase 5

```
"result"      { return RESULT_TK; }
"@pre"        { return PRE_TK; }
"@post"       { return POST_TK; }
"@invariant"  { return INV_TK; }
"@variant"    { return VAR_TK; }
"==>"        { return IMPLIES_TK; }
/* debe ir antes de la regla de ID */
[a-zA-Z]... { return ID_TK; }
```

Analizador sintáctico (Bison LALR(1))

```
Program : 'public' 'class' ID  
        '{' Members '}'
```

```
Method : 'public' 'static' Tipo  
        ID '(' ')' Block
```

```
Instr : Asig | Return | If  
      | 'while' '(' Expr ')',  
      OptInv OptVar Block
```

```
/* Anotaciones formales */
```

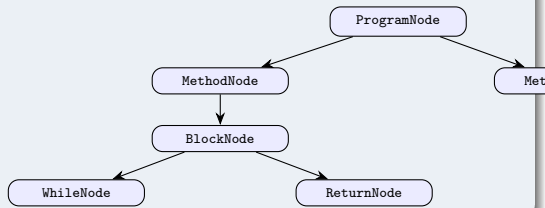
```
OptPre : '@pre' Expr | /*empty*/
```

```
OptPost : '@post' Expr | /*empty*/
```

```
OptInv : '@invariant' Expr | /*empty*/
```

```
OptVar : '@variant' Expr | /*empty*/
```

AST generado



Patrón Visitor

SemanticVisitor, IRBuilder, ASTVisualizer — todos implementan la interfaz Visitor sin tocar el AST

Tabla de símbolos y comprobaciones

TablaSimbolos

```
struct Simbolo {  
    string nombre;  
    unsigned tipo; // INT, REAL, BOOL  
    unsigned dir; // stack offset  
    bool isConstant;  
    bool isFunction;  
    int returnType;  
};  
class TablaSimbolos {  
    TablaSimbolos* padre; // scope  
    bool set(Simbolo s);  
    Simbolo* get(string n);  
};
```

Comprobaciones

- 1 Declaración antes de uso
- 2 Compatibilidad de tipos
- 3 No reasignación de final
- 4 Tipo de retorno correcto
- 5 Código inalcanzable
- 6 Llamadas a funciones válidas

Coerción de tipos

int op double $\Rightarrow$ double

int / int $\Rightarrow$ int

Cuadruplas de tres direcciones

Formato: (op, arg_1, arg_2, res)

IR_ADD/SUB/MUL/DIV aritmética

IR_MOD / IR_NEG módulo, negación

IR_COPY $x \leftarrow a$

IR_JZ / IR_JMP saltos
condicional/incond.

IR_LABEL etiqueta de destino

IR_CALL $t \leftarrow f()$

IR_RETURN devuelve valor

Ejemplo: while + suma

```
[FUNC_BEGIN] sumToFive
[LABEL] L0
[LTE] i 5 -> t0
[JZ] t0 -> L1
[ADD] s i -> t1
[COPY] t1 -> s
[ADD] i 1 -> t2
[COPY] t2 -> i
[JMP] L0
[LABEL] L1
[RETURN] s
[FUNC_END] sumToFive
```

Optimizaciones (−31 % instrucc.)

1. Elim. código muerto
2. Plegado constantes
3. Prop. de copias

Generación de código x86-64

ABI System V AMD64

Argumentos %rdi,%rsi,%rdx,... %r9

Retorno int %rax

Retorno float %xmm0

Caller-saved %rcx,%rdx,%r8... %r11

```
; prologo / epilogo estandar
pushq %rbp
movq %rsp, %rbp
subq $N, %rsp ; N % 16 == 0
...
movq %rbp, %rsp
popq %rbp
ret
```

Bucle compilado

```
sumToFive:
    pushq %rbp
    movq %rsp, %rbp
    subq $48, %rsp
L0:
    movq -8(%rbp), %rax ; i
    cmpq $5, %rax
    setle %al
    movzbl %al, %rax
    movq %rax, -24(%rbp) ; t0
    testq %rax, %rax
    je L1
    ; s = s + i
    movq -16(%rbp), %rax
    addq -8(%rbp), %rax
    movq %rax, -16(%rbp)
    jmp L0
L1: movq -16(%rbp), %rax
    ...
    ret
```

Chaitin-Briggs: coloración de grafos ($K = 6$)

Tres fases

1 Vivacidad (punto fijo)

$$\text{live_in}[B] = \text{use}[B] \cup (\text{live_out}[B] \setminus \text{def}[B])$$

2 Grafo de interferencias

Arista (d, v) si d se define con v viva

3 Coloración

- *Simplify*: apila nodos grado $< K$
- *Spill*: derrama el de mayor grado
- *Assign*: asigna colores al desapilar

Registros disponibles

Enteros: %rcx, %rdx, %r8
 %r9, %r10, %r11
Floats: %xmm2... %xmm7

Resultado

```
$ ./compiler --reg-alloc t04.java
i  -> %rcx
s  -> %rdx
t0 -> %r8
t1 -> spill (-32(%rbp))
```

Lógica de Hoare y cálculo WP

Tripleta de Hoare

$$\{P\} S \{Q\}$$

Si P vale antes de S y termina, Q vale después.

Condición principal:

$$P \Rightarrow \text{WP}(\textit{body}, Q)$$

válida $\iff$ Z3 dice UNSAT a $\neg VC$

Reglas del cálculo WP

$$\text{WP}(x := e, Q) = Q[e/x]$$

$$\text{WP}(S_1; S_2, Q) = \text{WP}(S_1, \text{WP}(S_2, Q))$$

$$\text{WP}(\text{ret } e, Q) = Q[e/\text{result}]$$

$$\begin{aligned} \text{WP}(\text{if}(B) S_1 \text{ else } S_2, Q) = & (B \Rightarrow \text{WP}(S_1, Q)) \\ & \wedge (\neg B \Rightarrow \text{WP}(S_2, Q)) \end{aligned}$$

Implementación CPS para retornos anticipados

Problema con la regla naïve

Para `if(x<0) return -x; return x;` con $Q = \text{result} \geq 0$: la composición inversa pierde `result` tras el primer `return`.

Solución: continuaciones (procesado hacia adelante)

```
Predicate* wpSeq(stmts, idx, Q) {
    auto* s = stmts[idx];
    // return: usa Q directamente -- ignora continuacion
    if (auto* r = dyn_cast<ReturnNode>(s))
        return wpReturn(r, Q);
    // if: la continuacion se propaga a ambas ramas
    if (auto* i = dyn_cast<IfNode>(s)) {
        vector<StmtNode*> cont(stmts.begin()+idx+1, stmts.end());
        return And(B ==> wpBranchAndCont(then, cont, Q),
                   !B ==> wpBranchAndCont(else, cont, Q));
    }
    // assign: WP(x:=e; resto, Q) = WP(resto, Q)[e/x]
    if (auto* a = dyn_cast<AssignNode>(s))
        return wpAssign(a, wpSeq(stmts, idx+1, Q));
    return wpSeq(stmts, idx+1, Q);
}
```

VCs de bucles con invariante

Para $\text{while}(B)\{S\}$ con invariante I :

Tres condiciones de verificación

VC_1 (**iniciación**):

$$P \Rightarrow \text{WP}(S_{\text{antes}}, I)$$

VC_2 (**preservación**):

$$(I \wedge B) \Rightarrow \text{WP}(S_{\text{body}}, I)$$

VC_3 (**uso**):

$$(I \wedge \neg B) \Rightarrow \text{WP}(S_{\text{desp}}, Q)$$

El invariante debe ser *fuerte*

Para la suma $s = 1 + \dots + i$:

$I = s \geq 0$ insuficiente:

Z3 halla $(i = -1, s = 0)$ que satisface
 $I \wedge B$

pero $(s + i) = s - 1 < 0$. **Fallo.**

$I = s \geq 0 \wedge i \geq 1$ correcto:

$$(s \geq 0 \wedge i \geq 1 \wedge i \leq 5) \Rightarrow (s + i) \geq 0$$

Z3: UNSAT ✓

SMT-Lib2 y Z3

```
1 (push 1)
2 ; [absValue] correccion
3 (declare-fun x () Int)
4 (assert (not
5   (= true
6     (and
7       (= (< (- 5) 0)
8         (>= (- (- 5)) 0))
9       (= (not (< (- 5) 0))
10         (>= (- 5) 0))
11     )
12   ))
13 (check-sat) ; --> unsat [OK]
14 (pop 1)
```

Interpretación de Z3

unsat VC válida [OK]

sat Bug hallado [BUG]

unknown Timeout [?]

Tipos de VC generadas

- 1 Corrección del método
- 2 Iniciación del invariante
- 3 Preservación del invariante
- 4 Uso del invariante

Demo: verificación completa

```
1 @pre true
2 @post result >= 0
3 public static int absValue() {
4     int x; x = -5;
5     if (x < 0) return -x;
6     return x;
7 }
8
9 @post result > 10 // BUG!
10 public static int wrongMethod() {
11     int x; x = 5; return x;
12 }
13
14 @post result >= 0
15 public static int sumLoop() {
16     int i; int s; i=1; s=0;
17     while (i <= 5)
18     @invariant s>=0 && i>=1
19     @variant 6 - i
20     { s=s+i; i=i+1; }
21     return s;
22 }
```

Salida del verificador

```
$ ./compiler --verify demo.java
```

```
[OK]    absValue
        correccion (unsat)

[OK]    sumLoop
        iniciacion (unsat)
[OK]    sumLoop
        preservacion (unsat)
[OK]    sumLoop
        uso (unsat)
[OK]    sumLoop
        correccion (unsat)
[OK]    sumLoop
        terminacion (unsat)

[BUG]   wrongMethod
        correccion (sat) <-- bug!
```

Suite de pruebas: 11/11

Backend (5)

t01_print	PASS
t02_arith	PASS
t03_if_else	PASS
t04_while	PASS
t05_funcs	PASS

Semántico (3)

sem01_undecl	PASS
sem02_type	PASS
sem03_redecl	PASS

Verificación (3)

ver01_simple	PASS
ver02_inv	PASS
demo_verified	PASS

```
make && tests/run_tests.sh ⇒ 11 /  
11 passed
```


Resumen de hitos

Fase	Milestone	Componentes clave	
1	Front-end	Flex + Bison, AST C++	✓
2	Semántico	TablaSimbolos, tipos, Visitor	✓
3	IR + Optim.	Cuadruplas, dead code, folding, copy	✓
4	Backend	x86-64, ABI, Chaitin-Briggs	✓
5	Verificación	Hoare, WP-CPS, Z3, VCGenerator	✓

Aportaciones principales

- Compilador completo end-to-end
- WP con continuaciones (CPS) para retornos anticipados
- VCs automáticas: corrección, invariantes, terminación
- Integración Z3 con reporte coloreado

Trabajo futuro

- Inferencia automática de invariantes (Daikon)
- Soporte de arrays y objetos
- SSA form para optimización avanzada
- Backend LLVM-IR

Gracias

Compilador JavaSubset $\rightarrow$ x86-64
con Verificación Formal mediante Hoare + Z3

`make && tests/run_tests.sh` $\Rightarrow$ **11/11 passed**
`./compiler -verify demo.java` $\Rightarrow$ **6 OK, 1 BUG**
`pdflatex presentation.tex` $\Rightarrow$ **estas diapositivas**